

الدليل الشامل لمشروع Tavily MCP Server (بالعربية)

هذا الدليل مكتوب لتفهم المشروع من الصفر: ما الفكرة، ما المشكلة، كيف حُلَّت، ما التقنيات، ما الـ API، وماذا يفعل كل أمر استُخدم أثناء العمل. للنسخة الإنجليزية: [GUIDE.en.md](#)

الفهرس

- [1. فكرة المشروع ومن أين جاءت](#)
- [2. المشكلة والحل](#)
- [3. ما هو MCP وكيف يعمل المشروع](#)
- [4. التقنيات المستخدمة](#)
- [5. واجهة Tavily API](#)
- [6. هيكل الملفات وشرح الكود](#)
- [7. الأدوات الأربع وأمثلة تجربتها](#)
- [8. شرح كل أمر استُخدم](#)
- [9. الرفع إلى GitHub: ما نسمح به وما نمنعه](#)
- [10. مشاكل واجهتنا وديروس مستفادة](#)
- [11. الأمان وإدارة المفتاح](#)

1. فكرة المشروع ومن أين جاءت

نماذج الذكاء الاصطناعي تتعلم من بيانات حتى تاريخ معين (ما يسمى *knowledge cutoff*)، ولا تملك بذاتها اتصالاً حياً بالإنترنت. فإذا سألتها عن خبر اليوم أو سعر حالي أو صفحة جديدة، قد تجيب من ذاكرتها القديمة.

الفكرة: نعطي المساعد أداة بحث حي في الويب عبر خادم صغير يتصل بخدمة **Tavily**، وهي خدمة بحث مصممة أصلاً لتطبيقات الذكاء الاصطناعي (تُرجع محتوى نظيفاً مرتباً بدل صفحات HTML خام).

ملاحظة صريحة: هذا القسم يشرح الدافع العام للمشروع كما يظهر من كوده وأهدافه (بحث حي داخل عميل MCP المكتبي و عميل MCP عبر الطرفية). لا يوجد في ملفات المشروع نص يوثق قصة بدايته، فإن كانت لك قصة شخصية للبداية فأضفها هنا.

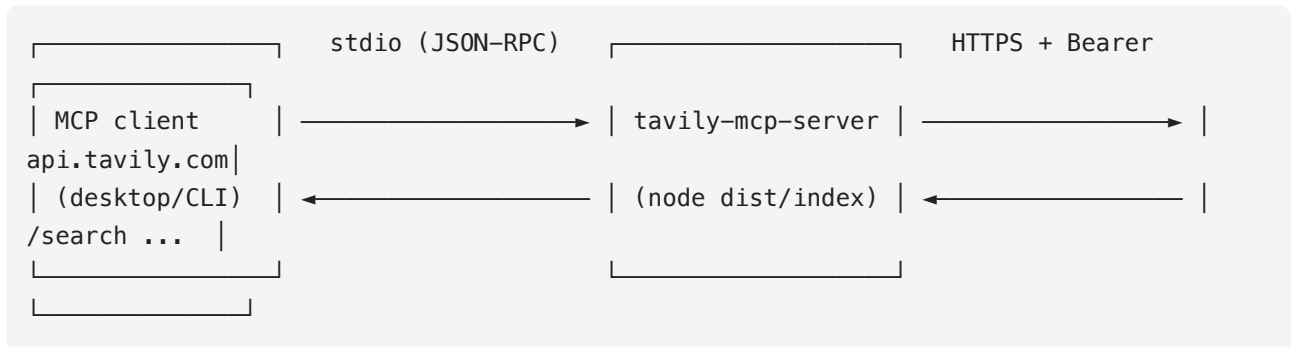
2. المشكلة والحل

المشكلة	الحل في المشروع
النموذج لا يرى الإنترنت ومعلوماته قديمة	أداة <code>tavily_search</code> تبحث حياً وتُرجع نتائج بمصادرها

المشكلة	الحل في المشروع
نتائج البحث العادية فيها ضجيج HTML	Tavily يُرجع نصاً نظيفاً مع درجة صلة (score)
نحتاج قراءة صفحة كاملة لا مقتطفاً	أداة <code>tavily_extract</code> تستخرج محتوى روابط محددة
نحتاج معرفة بنية موقع كامل	<code>tavily_map</code> يعطي الروابط، <code>tavily_crawl</code> يجلب المحتوى
لكل برنامج طريقة اتصال مختلفة بالأدوات	بروتوكول MCP يوحد طريقة التوصيل (عميل MCP المكتبي و عميل MCP عبر الطرفية و Cursor)
ردود ضخمة تستهلك سياق النموذج	<code>truncate</code> يقص الرد عند 25,000 حرف مع تنبيه
أخطاء الـ API غامضة	<code>explainHttpError</code> يحوّل أكواد الخطأ إلى رسائل مفهومة
تسريب المفتاح	المفتاح يُقرأ من متغير بيئة، و <code>env.</code> خارج Git

3. ما هو MCP وكيف يعمل المشروع

MCP (Model Context Protocol) بروتوكول مفتوح يسمح لتطبيقات الذكاء الاصطناعي (العميل) بالاتصال بأدوات خارجية (الخادم) بطريقة موحدة.



- التسلسل عند كل بحث: 1. عميل MCP المكتبي يشغّل الأمر `node dist/index.js` كعملية فرعية (من ملف الإعدادات).
- يتواصلان عبر **stdio** (الإدخال والإخراج القياسي) برسائل **JSON-RPC**: `initialize` ثم `tools/list` ثم `tools/call`. حين تطلب منه البحث، يرسل `tools/call` باسم `tavily_search`.
- خادماً يتحقق من المدخلات (zod)، ثم يرسل **POST** إلى `https://api.tavily.com/search` مع المفتاح.
- يحوّل الرد إلى Markdown (أو JSON) ويعيده إلى المساعد، وقد يقصّه إن طال.

مهم: رسائل السجل تُكتب على `( console.error ) stderr` لأن `stdout` محجوز لبروتوكول MCP، وكتابة أي شيء آخر عليه تقسد الاتصال.

4. التقنيات المستخدمة

التقنية	الإصدار (المثبت عندنا)	الدور
Node.js	x.22 (المطلوب 18 ≤)	بيئة تشغيل النسخة الرئيسية
TypeScript	x.5.9	لغة الكتابة، تُترجم إلى JavaScript في <code>/dist</code>
<code>modelcontextprotocol/sdk@</code>	x.1.32	مكتبة MCP الرسمية (<code>McpServer</code> و <code>StdioServerTransport</code>)
<code>zod</code>	x.3.25	التحقق من المدخلات وتوليد وصف المعاملات
<code>fetch</code> المدمج في Node	—	استدعاء Tavily (لا مكتبات HTTP إضافية)
<code>tsx</code>	x.4	تشغيل TypeScript مباشرة أثناء التطوير (<code>npm run dev</code>)
Python	3.12	النسخة البديلة البسيطة <code>server.py</code>
<code>mcp</code> (FastMCP)	> 2	مكتبة MCP للبايثون
<code>tavily-python</code>	≤ 0.5.0	عميل Tavily الرسمي للبايثون
<code>python-dotenv</code>	≤ 1.0.0	قراءة <code>env.</code> في نسخة بايثون
<code>uv</code>	—	إنشاء بيئة بايثون 3.12 بسرعة (نظام Mac عندنا كان 3.9)
Git و (<code>gh</code>) GitHub CLI	—	الإصدارات والرفع

لماذا نسختان؟ (TypeScript) `src/` هي الأشمل: أربع أدوات وتحقق دقيق وتنسيق مخرجات. `server.py` نسخة مبسطة بأداة واحدة `web_search` مفيدة للتعلم والمقارنة. المعتمدة في عميل MCP المكتبي الآن هي نسخة `TypeScript`.

5. واجهة Tavily API

- العنوان الأساسي: `https://api.tavily.com`
- الطريقة: `POST` بجسم JSON
- التوثيق: ترويسة `Authorization: Bearer tvly-...`
- الحصول على مفتاح: `https://app.tavily.com`
- انتهاء المهلة في كودنا: 180 ثانية (`AbortSignal.timeout(180_000)`)

نقطة النهاية	تُستخدم في الأداة	الوظيفة
<code>search/</code>	<code>tavily_search</code>	بحث في الويب

نقطة النهاية	تُستخدم في الأداة	الوظيفة
extract/	tavily_extract	استخراج محتوى من روابط (حتى 20)
map/	tavily_map	اكتشاف روابط موقع بلا محتوى
crawl/	tavily_crawl	الزحف على موقع مع جلب المحتوى

أهم معاملات `search:` `query` , `search_depth` (basic | advanced | fast | ultra-/ fast), `topic` (general | news | finance), `max_results` (1-20), `time_range` YYYY-MM-DD , بصيغة (day / week / month / year), `start_date` / `end_date` , `include_answer` , `include_raw_content` , `include_images` , `include_domains` / `exclude_domains` , `country` .

أكواد الأخطاء التي نترجمها:

الكود	المعنى	رسالتنا
400	معاملات غير صالحة	تحقق من قيم المعاملات
401	مفتاح خاطئ	تأكد أن المفتاح يبدأ بـ <code>-tvly</code>
429	تجاوز معدل الطلبات	انتظر وقلل الطلبات
433 / 432	تجاوز الخطة أو الرصيد	راجع الاستهلاك في <code>app.tavily.com</code>

الاستهلاك: `"search_depth="advanced` يكلف رصيدين (حسب وصف الكود). و `crawl` و `extract` المتقدم يستهلكان أكثر. راجع صفحة الأسعار الرسمية لتفاصيل الرصيد.

6. هيكل الملفات وشرح الكود

```
.
├── src/
│   ├── index.ts          # تعريف الخادم والأدوات الأربع
│   └── client.ts         # وأخطاؤه وقص الردود Tavily استدعاء
├── dist/                 # (Git لا يُرفع إلى) ناتج الترجمة
├── server.py             # (web_search أداة) النسخة البديلة بايثون
├── requirements.txt      # مكتبات بايثون
├── package.json          # npm وأوامر Node مكتبات
├── tsconfig.json         # TypeScript إعدادات مترجم
├── .env                  # المفتاح الحقيقي (لا يُرفع أبداً)
├── .env.example          # قالب بلا مفتاح (يُرفع)
├── .gitignore            # ما يُمنع رفعه
├── mcp_client_config.example.json # MCP مثال إعداد عميل
├── README.md / README.ar.md
└── docs/GUIDE.ar.md / GUIDE.en.md # هذا الدليل
```

src/client.ts

- `CHARACTER_LIMIT = 25000` و `BASE_URL`.
- `tavilyPost(endpoint, body)`: تقرأ `TAVILY_API_KEY` من `process.env` وترمي خطأ واضحاً إن غاب. تحذف القيم `undefined` من الجسم ليطبق Tavily افتراضياته. ترسل `POST` وترجم أي استجابة غير ناجحة عبر `explainHttpError`.
- `truncate(text)`: تقص النص عند 25,000 حرف وتضيف تنبيهاً يوجّه لتقليل النتائج.

src/index.ts

- ينشئ `McpServer` باسم `tavily-mcp-server`.
- `readOnly`: تلميحات (annotations) تخبر العميل أن الأدوات للقراءة فقط (`readOnlyHint`) وغير مدمرة، فلا تغيّر شيئاً عندك.
- `responseFormat`: خيار `markdown` (افتراضي، مقروء) أو `json` (الرد الخام).
- `ok` () تبني الرد (نص + `structuredContent`)، و `fail` () تُرجع `isError: true` برسالة مفهومة بدل إسقاط الخادم.
- لكل أداة: `registerTool` (اسم، {وصف، `inputSchema`، `outputSchema`، `zod`، `annotations`، دالة}).
- في الختام: `StdioServerTransport` ثم `server.connect(transport)`.

server.py (البديل)

أداة واحدة `web_search`. تتحقق من المفتاح ومن `query` و (1-20) `max_results` و `search_depth`. عميل Tavily متزامن، لذلك يُنفَّذ داخل `asyncio.to_thread` كي لا يحجب حلقة الأحداث. تُرجع JSON بـ `title` و `url` و `content` أو `{"error": "...}"`. تقرأ `env` من جانب الملف نفسه مهما كان مجلد التشغيل.

7. الأدوات الأربع وأمثلة تجربتها

ألصقها في محادثة جديدة في عميل MCP المكتبي، واذكر اسم الأداة حتى لا يستخدم المساعد البحث المدمج.

tavily_search - استخدم `tavily_search` للبحث عن "ما الجديد في TypeScript"، وأعطني 5 نتائج - استخدم `tavily_search` بـ `topic=news` و `time_range=week` للبحث عن أخبار الذكاء الاصطناعي، مع `include_answer=true` - استخدم `tavily_search` للبحث عن "MCP server examples" بشرط `include_domains=["github.com"]` و `max_results=3`

tavily_extract - استخدم `tavily_extract` على `https://modelcontextprotocol.io/introduction` ولخص المحتوى - ... مع `chunks_per_source=2` و `query="what is MCP"`

tavily_map (الأرخص: روابط فقط) - استخدم **tavily_map** على <https://modelcontextprotocol.io> واعرض لي الروابط

tavily_crawl (الأعلى، ابدأ بحدود صغيرة) - استخدم **tavily_crawl** على <https://modelcontextprotocol.io> مع `max_depth=1` و `limit=5`، ولخص الصفحات -
... مع `instructions="find pages about building servers"`

علامات النجاح: يظهر طلب إذن باسم الأداة، والنتائج فيها روابط حقيقية، و **map** ترجع روابط فقط بينما **crawl** ترجع روابط مع محتوى.

8. شرح كل أمر استخدم

مقسمة إلى: (أ) أوامر البناء الأساسية، (ب) أوامر هذه الجلسة بالترتيب.

(أ) أوامر البناء والتشغيل الأساسية

الأمر	ماذا يفعل
<code>npm install</code>	يقرأ <code>package.json</code> وينزل المكتبات إلى <code>/node_modules</code> ويكتب <code>package-lock.json</code> لتثبيت الإصدارات.
<code>npm run build</code>	ينفذ <code>tsc</code> (سكربت <code>build</code>) فيترجم <code>src/*.ts</code> إلى <code>dist/*.js</code> حسب <code>tsconfig.json</code> .
<code>npm start</code>	<code>node dist/index.js</code> لتشغيل الخادم يدوياً (ينتظر رسائل MCP على <code>stdin</code>).
<code>npm run dev</code>	<code>tsx watch src/index.ts</code> للتطوير: يعيد التشغيل عند تعديل الكود دون ترجمة.
<code>client-cli> mcp add tavily -e> TAVILY_API_KEY=... -- node .../dist/index.js</code>	يسجل الخادم في عميل MCP عبر الطرفية. - <code>e</code> يمرر متغير البيئة، وما بعد <code>--</code> هو أمر التشغيل.
<code>npx @modelcontextprotocol/inspector node dist/index.js</code>	يفتح واجهة رسمية لتجربة الأدوات يدوياً. وضع المفتاح قبله: <code>TAVILY_API_KEY=...</code> <code>.... npx</code>
<code>uv venv --python 3.12 .venv</code>	ينشئ بيئة بايثون معزولة بإصدار 3.12 (نظام Mac عندنا 3.9 وهو أقدم من المطلوب 3.10).

الأمر	ماذا يفعل
<code>uv pip install --python .venv/bin/python -r requirements.txt</code>	يثبت مكتبات بايثون داخل تلك البيئة فقط.
<code>venv/bin/mcp dev server.py.</code>	يشغل نسخة بايثون مع المفتش الرسمي.

ب) أوامر هذه الجلسة بالترتيب

(1) فحص حالة المشروع - `ls -la`: يعرض الملفات كلها بما فيها المخفية (`env.`) مع الصلاحيات والأحجام. - `find . -path ./node_modules -prune -o -type f -not -path './.git/*' -print | head -50`: يسرد الملفات مع تجاهل `node_modules` و `git.`، و `-50` يكتفي بأول 50 سطراً. - `cat .env`: يقرأ الملفات. الأمر `sed 's/=.*<hidden>/'` هنا يخفي قيمة المفتاح بعد علامة `=` حتى لا تظهر في المحادثة. - `wc -l src/*.ts`: يعد أسطر كل ملف.

(2) اختبار النسختين بشكل حقيقي - اختبار بايثون: `venv/bin/python - <<'EOF' ... EOF`. كود بايثون مكتوباً داخل الأمر (heredoc). استورد `server` واستدعى `web_search` عبر `asyncio.run` ببحث فعلي. النتيجة: نتيجتان بلا أخطاء. - اختبار `TypeScript: node -e '...'` يشغل سكريبتاً قصيراً يطلق `node dist/index.js` كعملية فرعية (`spawn`) ويتحدث معه بـ JSON-RPC يدوياً: `initialize` ثم `notifications/initialized` ثم `tools/list` ثم `tools/call`. هذا ما يفعله عميل MCP المكتبي بالضبط، فثبتنا أن الأدوات الأربع تظهر وأن البحث يعمل. - أول مرة رجع خطأ `TAVILY_API_KEY not set` لأن نسخة TS لا تقرأ `.env`. الحل: `set +a` و `set -a` ثم تشغيل الاختبار. شرحها: - `set a` يجعل كل متغير يُعرّف تالياً مُصدراً للعمليات الفرعية، و `env./.` يقرأ الملف في الجلسة الحالية، و `set +a` يوقف التصدير التلقائي.

(3) ربط عميل MCP المكتبي - `<client config file "ls -la">` وطباعته مع إخفاء أي حقل يحوي `KEY` أو `TOKEN` أو `SECRET` بـ `sed -E`: لنعرف ما فيه قبل التعديل. - `cp "$F" "$F.bak"`: نسخة احتياطية قبل أي تعديل. (احذفها لاحقاً لأن فيها المفتاح). - `node -e '...'` للدمج: يقرأ `env.` بتعبير نمطي ويستخرج المفتاح، ثم يقرأ ملف الإعدادات JSON، ويضيف `mcpServers.tavily`، ويكتبه من جديد. الدمج يحافظ على باقي إعداداتك بدل استبدال الملف. - `chmod 600 "$F"`: يجعل الملف مقروءاً ومكتوباً لك وحدك لأن فيه المفتاح الآن. - حذف نسخة بايثون من الإعدادات بسكريبت `node -e mcpServers["tavily-web-"]` آخر يحذف `search` ثم إعادة الفتح: لأن عميل MCP المكتبي يقرأ الإعدادات عند التشغيل فقط. `Cmd+Q`.

(4) استكشاف الأخطاء - `ls -t <client logs folder> | head` ثم `tail` على `mcp-server-tavily.log`: السجل أثبت أن الخادم بدأ وأرسل قائمة أدواته لكن بلا أي `tools/call`، أي أن المساعد لم يستخدم أدوات بل بحثه المدمج. فتحقق من ذلك بدل التخمين. `grep -v -i 'tvly'` يستبعد أي سطر قد يحوي مفتاحاً.

(5) تنظيف ملفات بايثون (ثم أُعيدت لاحقاً) - `du -sh .venv __pycache__` و `__cat .gitignore`: نراجع ما سنحذف ونعرف حجمه (42MB). - `rm -rf .venv __pycache__ server.py`: حذف نهائي بلا سلة مهملات، ولذلك راجعنا الهدف قبل التنفيذ. وقد أُعيدت الملفات بطلبك لاحقاً.

(6) فحص المفاتيح قبل الرفع - `--exclude` . `grep -rIlE 'tvly-[A-Za-z0-9_-]{10,}'` . `dir=node_modules`: يبحث في المجلد عن أي نص يشبه مفتاح Tavily ويطبّع أسماء الملفات فقط. (`r-` تكراري، - `I` يتجاهل الثنائي، `l-` أسماء فقط، `E-` تعبير نمطي.) - **فخ تعلمناه:** `I-` يعتبر الملف ثنائياً إن احتوى عربي في بعض إعدادات اللغة، فتجاهل `env`. (فيه تعليق عربي) ولم يظهره. استخدم `a-` بدلاً منه حين يحتوي الملف عربياً. - `grep -- "$K" rIlF`: يبحث عن نص المفتاح الحقيقي بالضبط (`F-` نص حرفي)، و `--` تمنع اعتبار المفتاح خيار أمر.

(7) الرفع إلى `git init -q -b main` - `GitHub`: ينشئ مستودعاً محلياً بفرع `main` (`q-` بلا ضجيج). - `git add -A`: يجهّز كل الملفات لا `commit` باستثناء ما في `git ls-files` . `gitignore`: يعرض ما سيُرفع فعلاً، وهي نقطة المراجعة الأهم. - `git grep -aF -- "$K"`: يبحث عن المفتاح الحقيقي داخل الملفات المتتبعة فقط؛ النتيجة المطلوبة "لا يوجد". - `gh auth status`: يتأكد أنك مسجّل الدخول في `GitHub CLI`. - `git commit -m "..."`: يسجّل اللقطة الأولى. - `gh repo create tavily-mcp-server --private`. - `--push`، `--source=.`: ينشئ مستودعاً خاصاً على حسابك، ويربط المجلد الحالي به (`source=.`)، ويرفع (`push-`). - `git log --oneline` و `git status -sb`: يتأكدان من وجود ال `commit` وتطابق الفرع المحلي مع البعيد. - أمر لاحق: `sed -i '' 's#...#...#' README.md` يستبدل نصاً داخل الملف في `macOS` (ال `'` مطلوبة ل `sed` على `Mac`). استبدلنا مساري الشخصي بمسار عام.

9. الرفع إلى GitHub: ما نسمح به وما نمنعه

✓ مسموح بالرفع

السبب	الملف/المجلد
الكود المصدري	<code>/src</code>
يحددان المكتبات والإصدارات لإعادة البناء	<code>package.json</code> و <code>package-lock.json</code>
إعداد الترجمة	<code>tsconfig.json</code>
النسخة البديلة	<code>requirements.txt</code> و <code>server.py</code>
قالب بلا مفتاح حقيقي	<code>env.example</code> .
مثال بمسارات عامة ومفتاح وهمي	<code>mcp_client_config.example.json</code>
التوثيق	<code>README.md</code> و <code>README.ar.md</code> و <code>/docs</code>
يحمي من رفع الأسرار	<code>gitignore</code> .

✗ ممنوع الرفع

السبب	الملف/المجلد
فيه المفتاح الحقيقي	<code>env</code> .

السبب	الملف/المجلد
قد يحوي المفتاح	mcp_client_config.json (الحقيقي)
نسخ إعدادات فيها المفتاح	bak.*
ضخم ويُعاد تنزيله بـ npm install	/node_modules
ناتج ترجمة يُعاد بـ npm run build	/dist
بيئة بايثون (42MB) تُعاد بـ uv venv	/venv.
ملفات مؤقتة من النظام	DS_Store. و /__pycache__

قائمة تحقق قبل كل git push

```
git status # bak ولا .env لا يظهر
git ls-files # راجع القائمة كاملة
K=$(grep -E '^TAVILY_API_KEY=' .env | cut -d= -f2- | tr -d '"' ' ')
git grep -aF -- "$K" && echo "نظيف" || echo "خطأ: المفتاح داخل ملف متتبع"
```

إن كان المفتاح قد دخل في commit سابق فحذفه من الملفات لا يكفي (يبقى في التاريخ). الحل الوحيد الموثوق: إبطال المفتاح من app.tavily.com وإنشاء غيره.

10. مشاكل واجهتنا ودروس مستفادة

المشكلة	السبب	الحل
TAVILY_API_KEY not set عند اختبار TS	نسخة TS لا تقرأ .env	تمرير المفتاح عبر env في الإعدادات أو set -a; . ./env
المساعد لم يطلب إذنًا ولم يستخدم Tavily	استعمل البحث المدمج	محادثة جديدة، وتسمية الأداة في الطلب، أو تعطيل البحث المدمج
ال خادم لا يظهر بعد التعديل	عميل MCP المكتبي يقرأ الإعدادات عند التشغيل	Cmd+Q ثم إعادة الفتح
python3.12 not found	النظام فيه 3.9 فقط	uv venv --python 3.12 ينزله ويجهزه
grep -I تجاهل .env	محتوى عربي اعتُبر ثنائياً	استخدام -a
تعديل ملف الإعدادات قد يمسح إعدادات أخرى	الملف فيه إعدادات التطبيق كلها	دمج بسكريبت بدل الاستبدال + نسخة bak.

11. الأمان وإدارة المفتاح

- المفتاح يُقرأ من `process.env.TAVILY_API_KEY` ولا يُكتب في الكود.
- لا تضع المفتاح في أي ملف يُرفع إلى Git.
- ملف إعدادات عميل MCP المكتبي يحوي المفتاح، وصلاحياته `600` (لك وحدك).
- الأدوات كلها للقراءة فقط (`readOnlyHint`)، ولا تكتب شيئاً على جهازك.
- لإبطال مفتاح: Delete ← API Keys ← <https://app.tavily.com> أو Regenerate، ثم حدّث `env`. وإعدادات عميل MCP المكتبي.